ADDIS ABABA INSTITUTE OF TECHNOLOGY
School of Electrical and Computer Engineering
Department of Computer Science and Engineering

Retail Checkout Automation System Using Computer Vision

Design Document

Capstone Engineering Project — Final Report

Submitted by:

[Student Full Name]
Student ID: [XXXX/XX]

Supervised by:

[Supervisor Full Name], [Title]
Department of Computer Science and Engineering

Project Duration: [Start Month Year] — [End Month Year]
Submission Date: [Day Month Year]

CERTIFICATE OF COMPLETION / APPROVAL

This is to certify that the capstone project entitled *"Retail Checkout Automation System Using Computer Vision"* was prepared by [Student Full Name], Student ID [XXXX/XX], of the Department of Computer Science and Engineering, Addis Ababa Institute of Technology, in partial fulfilment of the requirements for the degree of Bachelor of Science in Computer Science and Engineering. The project has been submitted with our approval.

Supervisor: \[Supervisor Full Name, Title\]: _____

Date: _____

Project Coordinator / Capstone Chair: _____

Date: _____

Department Head: _____

Date: _____

ABSTRACT

Manual checkout in Ethiopian supermarkets introduces measurable operational inefficiencies: cashiers must process each product individually through barcode scanning, handle damaged or absent barcodes through manual lookup, and issue receipts under high-throughput conditions. These constraints reduce transaction throughput, increase cashier cognitive load, and lengthen customer wait times during peak periods. Fully automated checkout systems — while effective in high-resource environments — remain impractical for local retail deployment due to prohibitive hardware, installation, and ongoing maintenance costs.

This report presents the design of a low-cost, cashier-assisted retail point-of-sale automation system using computer vision. The system is intended to accelerate receipt preparation for selected Ethiopian supermarket products by detecting items placed on a checkout table and generating a draft transaction for cashier review. A YOLOv8-based object detection model identifies visible packaged products; the backend matches detections against a product catalogue, resolves quantities, and retrieves current prices. The cashier retains authority over the final transaction, with explicit correction, manual entry, and barcode-fallback workflows supported throughout.

Hardware components include a fixed-position camera, digital weighing scale, barcode scanner, and a local computer or Raspberry Pi for prototype deployment. Software components include the YOLOv8 detection model, a Django REST backend, a React cashier interface, a product catalogue, Odoo ERP integration for product and transaction management, a payment simulation module, and a digital receipt generation workflow. A dedicated weighted-product module handles price-by-weight items such as fruits and vegetables.

System evaluation addresses two concerns. Model performance is assessed using precision, recall, mAP@50, mAP@50-95, confusion matrix analysis, and per-class detection speed. End-to-end system validation confirms correct product matching, duplicate counting, price retrieval, receipt total accuracy, barcode-fallback behaviour, weight-based pricing, and cashier correction workflow integrity. Together, these metrics determine whether the system provides a reliable and practical improvement over the current manual process.

ACKNOWLEDGEMENTS

[This section is optional. If included, acknowledge your supervisor, department staff, any organisations that provided product samples or access for data collection, and any funding or equipment support received.]

TABLE OF CONTENTS

Contents

1 Executive Summary 1
2 Introduction 1
2.1 Background and Context 1
2.2 Engineering Context and Motivation 2
2.3 Problem Statement 2
2.4 Objectives 2
2.4.1 General Objective 2
2.4.2 Specific Objectives 2
2.5 Scope 3
2.6 Report Structure 3
3 Requirements and Constraints 3
3.1 Functional Requirements 3
3.2 Non-Functional Requirements and Engineering Constraints 4
3.3 Success Criteria 4
4 System Design and Architecture 4
4.1 System Overview 4
4.2 Checkout Workflows 5
4.2.1 Normal Product Checkout 5
4.2.2 Weighted Product Checkout 5
4.3 Module Design 6
5 Materials and Methods 7
5.1 Hardware Components 7
5.2 Software Stack 7
5.3 Dataset Collection and Preparation 7
5.3.1 Product Selection 7
5.3.2 Image Capture 7
5.3.3 Annotation 8
5.3.4 Dataset Split 8
5.4 Engineering Calculations 8
5.4.1 Weighted Item Price Calculation 8
5.4.2 Receipt Total Calculation 8
6 Implementation 8
6.1 Hardware Deployment Layout 8
6.2 Detection Model Training 9
6.3 Backend Implementation 9
6.4 Cashier Interface 9
6.5 Odoo ERP Integration 9
6.6 Source Code Organisation 9

7 Testing and Validation 10
7.1 Test Strategy 10
7.2 Model Evaluation 10
7.3 System Validation 11
8 Results and Discussion 11
8.1 Model Performance Results 11
8.2 System Validation Results 12
8.3 Discussion 12
9 Conclusion and Recommendations 12
9.1 Summary 12
9.2 Achievement of Objectives 12
9.3 Limitations 12
9.4 Recommendations 13
J. References 13
K. Appendix A: Bill of Materials 14
L. Appendix B: Project Timeline 14
M. Appendix C: Test Logs 14

LIST OF FIGURES

Note: All figures are placeholders. Insert diagrams during implementation.

LIST OF TABLES

Table 1 List of abbreviations and acronyms viii
Table 2 Functional requirements 3
Table 3 Non-functional requirements and engineering constraints 4
Table 4 Success criteria 4
Table 5 Module responsibility summary 6
Table 6 Hardware bill of materials 7
Table 7 Dataset split summary 8
Table 8 System-level validation test cases 11
Table 9 Model performance summary (to be completed after training) 11
Table 10 Hardware bill of materials 14
Table 11 Indicative project Gantt chart (● = active period) 14

LIST OF ABBREVIATIONS AND ACRONYMS

Abbreviation	Expansion
API	Application Programming Interface
ERP	Enterprise Resource Planning
mAP	Mean Average Precision
OCR	Optical Character Recognition
POS	Point of Sale
REST	Representational State Transfer
SKU	Stock Keeping Unit
YOLO	You Only Look Once

Table 1: List of abbreviations and acronyms

1 Executive Summary

This report describes the design of a computer vision-based checkout assistance system for Ethiopian supermarkets. The system addresses inefficiencies in manual barcode-scanning workflows by using a camera and a YOLOv8 object detection model to identify packaged products placed on a checkout table. The detection results are passed to a Django backend, which matches them against product records in an Odoo ERP system and presents a draft receipt to the cashier through a React interface.

The cashier retains full transactional authority. The system is designed as an assistant, not a replacement: every detected item can be confirmed, corrected, or removed, and a barcode fallback provides a reliable alternative when vision-based detection fails. Weighted products — sold by kilogram rather than by unit — are handled through a separate workflow involving a digital scale and a dedicated price-by-weight calculation path.

The prototype is designed for low-cost deployment using a fixed camera, a barcode scanner, a digital weighing scale, and a local computer or single-board computer such as a Raspberry Pi. Testing covers both model-level metrics (precision, recall, mAP@50, mAP@50-95) and end-to-end system validation (product matching accuracy, receipt correctness, fallback reliability, cashier correction workflow).

The primary recommendation is that the system be evaluated on a defined set of common Ethiopian supermarket SKUs before extending coverage. Future iterations should incorporate confidence-based routing, human-in-the-loop dataset improvement, and low-cost edge deployment optimisation.

2 Introduction

2.1 Background and Context

Supermarket checkout is a high-frequency, high-throughput operational process. In most Ethiopian retail environments, this process remains entirely manual: each product is scanned individually, missing or damaged barcodes require manual lookup, and receipts are generated line by line. Under normal load, this workflow is manageable. Under peak-hour demand — when queues form and cashier throughput becomes a bottleneck — the limitations become commercially significant.

Automated checkout technologies exist and have been deployed at scale by large international retailers. These systems use a combination of weight sensors, multiple overhead cameras, RFID readers, and substantial backend infrastructure. Their cost, complexity, and maintenance requirements make them unsuitable for the majority of Ethiopian supermarkets, which operate in cost-constrained environments and lack the technical support infrastructure required for fully automated systems.

A more viable path is incremental improvement to the existing cashier workflow. Rather than removing the cashier, a computer vision system can assist by pre-identifying products and generating a draft receipt, reducing the per-item scanning burden. The cashier corrects and confirms the result, keeping a human in the loop while reducing the repetitive workload.

2.2 Engineering Context and Motivation

The motivation for this project is grounded in observable checkout problems:

- Customer queues lengthen during peak hours due to slow per-product processing.
- Cashier workload is repetitive and susceptible to error under time pressure.
- Damaged, missing, or incorrectly printed barcodes cause delays and require manual intervention.
- Transactions involving many items or mixed packaged and weighted products are disproportionately slow.
- No affordable, locally-validated checkout automation tool exists for Ethiopian retail products.

This project addresses these problems by designing a prototype system using commodity hardware (camera, scale, barcode scanner) and open-source software (YOLOv8, Django, React, Odoo), keeping both capital and operational costs low.

2.3 Problem Statement

The current manual checkout process requires the cashier to scan or enter every product individually. This constrains throughput, increases error rates under high load, and provides no mechanism for automated product identification. The result is longer customer waiting times, higher cashier workload, and reduced effective capacity during peak periods.

The engineering problem is: how can product identification and receipt preparation be partially automated in a low-cost, cashier-assisted configuration that is practical for Ethiopian supermarkets, tolerates model errors gracefully, and does not require replacement of existing cashier infrastructure?

2.4 Objectives

2.4.1 General Objective

To design and prototype a low-cost, cashier-assisted retail checkout automation system using computer vision that reduces the manual product-entry burden and accelerates receipt generation in Ethiopian supermarkets.

2.4.2 Specific Objectives

1. Collect and annotate a dataset of selected Ethiopian supermarket products under checkout-representative conditions.
2. Train and evaluate a YOLOv8 object detection model on the collected dataset.
3. Develop a Django backend that receives detection results, matches them against a product catalogue, resolves quantities, and manages checkout state.
4. Integrate Odoo ERP to provide authoritative product pricing and transaction record management.
5. Build a React cashier interface that supports detection review, correction, barcode fallback, and receipt confirmation.
6. Implement a weighted-product workflow using a digital scale and price-by-weight calculation.
7. Validate system correctness through end-to-end checkout tests covering detection accuracy, receipt accuracy, fallback reliability, and cashier correction workflow.

2.5 Scope

The project covers the design, implementation, and testing of a prototype checkout assistance system. The prototype targets a defined set of packaged supermarket products common in Ethiopian retail. Weighted-product support is included for fruits and vegetables. Payment in the prototype is simulated; live payment provider integration is outside the scope of this work. The system is not designed as a fully autonomous checkout terminal — human cashier involvement is an architectural requirement, not a limitation.

2.6 Report Structure

Section 3 specifies functional and non-functional requirements and engineering constraints. Section 4 describes the system architecture and module design. Section 5 covers dataset collection, materials, and methods. Section 6 describes prototype implementation. Section 7 presents the test plan and results. Section 8 analyses results and discusses performance. Section 9 states conclusions and recommendations. Appendices provide supporting material including code listings, bill of materials, and test logs.

3 Requirements and Constraints

3.1 Functional Requirements

ID	Requirement	Description
FR-01	Product detection	The system shall detect and classify visible packaged products in the camera image using a YOLO-based model.
FR-02	Duplicate counting	The system shall count repeated instances of the same product class and represent them as a single line item with quantity.
FR-03	Product matching	Detected product classes shall be matched against ERP product records to retrieve official names and prices.
FR-04	Cashier review	The cashier interface shall display detected items with quantities and prices, and allow confirmation, correction, removal, and addition of items.
FR-05	Barcode fallback	The system shall support barcode scanning or manual search as a fallback for any product the vision model fails to identify correctly.
FR-06	Weighted products	The system shall support price-by-weight items through a dedicated workflow using a digital scale and unit-price retrieval.
FR-07	Receipt generation	After cashier confirmation, the system shall generate a digital receipt containing product names, quantities, unit prices, subtotals, total, date, time, and payment status.
FR-08	Payment handling	The payment module shall record payment status (completed, pending, failed, or cancelled). Payment may be simulated in the prototype.
FR-09	Transaction record	Confirmed transactions shall be saved in the ERP system with a complete audit trail.
FR-10	Admin management	An administrator interface shall allow product creation, price update, category management, and transaction history review.

Table 2: Functional requirements

3.2 Non-Functional Requirements and Engineering Constraints

Category	Requirement	Target / Constraint
Performance	Detection latency	End-to-end image capture to draft receipt display: ≤ 3 s on prototype hardware.
Performance	Detection accuracy	mAP@50 ≥ 0.80 on held-out test set over trained product classes.
Cost	Hardware budget	Total prototype hardware cost shall be achievable using commodity components (camera, scale, scanner, local PC or SBC).
Reliability	Fallback coverage	Every product in the catalogue shall be reachable through barcode fallback regardless of vision model confidence.
Usability	Cashier interface	The cashier interface shall require no more than three interactions to correct, add, or remove any item.
Safety	Billing accuracy	The system shall never generate a confirmed receipt without explicit cashier confirmation. No item shall be billed without cashier review.
Scalability	Product catalogue	The backend shall support product catalogue expansion without requiring model retraining for non-vision-critical attributes.

Table 3: Non-functional requirements and engineering constraints

3.3 Success Criteria

ID	Criterion	Acceptable Threshold
SC-01	Object detection mAP@50	≥ 0.80 on held-out test images
SC-02	Receipt total accuracy	100 % correct totals for all confirmed-checkout test cases
SC-03	Barcode fallback reliability	100 % of test products reachable via barcode or manual search
SC-04	Weighted-item price accuracy	Price calculation error ≤ 0.01 ETB across all weight-based test cases
SC-05	End-to-end latency	Draft receipt displayed within 3 s of image capture on prototype hardware

Table 4: Success criteria

4 System Design and Architecture

4.1 System Overview

The system is designed around the physical checkout workflow. A customer places products on the checkout table; a fixed-position camera captures the scene; a YOLO-based model identifies visible products; a Django backend maps detections to ERP product records; the cashier reviews and confirms the draft receipt; and the confirmed transaction is saved. This pipeline is designed to be interruptible at any stage by cashier intervention.

[Insert Figure 4.1: System architecture block diagram here.]

4.2 Checkout Workflows

4.2.1 Normal Product Checkout

[Insert Figure 4.2: Normal product checkout flowchart here.]

1. The customer places packaged products on the checkout table.
2. The camera captures the scene from a fixed overhead or angled position.
3. The YOLO detection model processes the image and returns bounding boxes, class labels, and confidence scores.
4. The backend groups repeated class labels into quantities and filters low-confidence detections for cashier attention.
5. Detected items are matched against the Odoo ERP product catalogue to retrieve official product names and prices.
6. The cashier interface displays the draft receipt.
7. The cashier reviews each line item: confirms correct detections, removes wrong detections, adjusts quantities, and adds missed products.
8. For products not confidently identified, the cashier activates barcode fallback.
9. The backend recalculates subtotals and total after each correction.
10. The cashier confirms the final receipt.
11. The payment module records payment status.
12. The transaction is saved in the ERP system and a digital receipt is generated.

4.2.2 Weighted Product Checkout

[Insert Figure 4.3: Weighted product checkout flowchart here.]

1. The cashier places the weighted item (e.g., fruit or vegetable) on the digital scale.
2. The camera captures the item and, where possible, the scale display.
3. The weighted product recognition module identifies the product type from the camera image.
4. The scale reading module reads the displayed weight via OCR or, if OCR is unreliable, prompts the cashier for manual weight entry.
5. The backend retrieves the unit price per kilogram from the ERP product record.
6. The item price is calculated as unit price × measured weight.
7. The cashier confirms the calculated price.
8. The item is appended to the active checkout list as a line item.

4.3 Module Design

Module	Primary Responsibilities
Camera Input	Captures images of checkout table; provides input to detection model; must maintain fixed position and acceptable lighting.
Product Detection	YOLOv8 inference on captured frame; returns class labels, bounding boxes, confidence scores, and repeat counts.
Backend Checkout	Receives detection results; maps to ERP records; manages checkout state; applies cashier corrections; calculates totals; routes confirmed data to receipt and ERP modules.
ERP Integration	Odoo integration for product catalogue (names, barcodes, prices, categories), transaction storage, receipt workflow, and payment status tracking.
Cashier Interface	React interface for draft receipt review, item correction, barcode fallback activation, receipt preview, and transaction confirmation.
Barcode Fallback	Accepts barcode scan or manual product search; resolves to ERP product record; appends to checkout list.
Weighted Product	Handles price-by-weight workflow; identifies product type; receives weight; calculates item price from unit price × weight.
Scale Reading	Reads scale display via OCR or prompts cashier for manual weight entry as fallback.
Receipt Generation	Generates digital receipt from confirmed checkout data via Odoo ERP workflow; includes product names, quantities, prices, totals, timestamp, and payment status.
Payment	Records payment status; simulated in prototype; designed for future integration with Telebirr or Chapa.
Admin Management	Product creation, price update, category management, barcode assignment, transaction history review.

Table 5: Module responsibility summary

5 Materials and Methods

5.1 Hardware Components

Component	Role	Notes
Camera (USB / phone)	Image capture of checkout table	Fixed mounting required
Local PC or Raspberry Pi	Inference, backend, and interface hosting	GPU optional but beneficial
Digital weighing scale	Weight measurement for price-by-weight items	Display readable by camera
Barcode scanner (USB/BT)	Barcode fallback product lookup	HID-compatible preferred
Checkout table	Product placement surface	Plain background preferred
Display monitor	Cashier interface rendering	Touchscreen optional
Receipt printer (optional)	Physical receipt output	Digital receipt is primary

Table 6: Hardware bill of materials

5.2 Software Stack

The software stack is selected for compatibility, open-source availability, and practical deployability:

- **Detection model:** YOLOv8 (Ultralytics). Chosen for its balance of detection speed and accuracy, active maintenance, and straightforward training pipeline.
- **Backend framework:** Django with Django REST Framework. Provides a structured API layer between the detection module, ERP integration, and cashier interface.
- **Frontend framework:** React. Provides a responsive, component-based cashier interface with real-time draft receipt state management.
- **ERP system:** Odoo Community Edition. Manages the product catalogue, pricing, transaction records, and receipt generation workflow.
- **Database:** PostgreSQL (via Odoo) for product and transaction data.
- **OCR (optional):** Tesseract or equivalent for scale display reading.
- **Operating system:** Ubuntu 22.04 LTS or Raspberry Pi OS for prototype deployment.

5.3 Dataset Collection and Preparation

The object detection model requires a labelled dataset of supermarket products captured under conditions representative of the actual checkout environment.

5.3.1 Product Selection

A set of product classes is defined based on commonly available packaged goods in Ethiopian supermarkets. Selection criteria include visual distinctiveness, commercial prevalence, and variety of packaging shapes, sizes, and materials. A target of 20–40 initial product classes is recommended for the prototype.

5.3.2 Image Capture

Images are captured using the same camera and physical setup as the deployment environment. Capture conditions include:

- Varied lighting: natural daylight, overhead fluorescent, and mixed conditions.
- Single-product and multi-product arrangements, including partial occlusion.
- Multiple product orientations and positions on the table surface.
- Varied backgrounds within the checkout table area.

A minimum of 100 images per product class is recommended before annotation, with augmentation applied during training to expand effective sample size.

[Insert Figure 5.1: Sample annotated training images here.]

5.3.3 Annotation

Bounding box annotations are created using Roboflow or an equivalent tool. Each product instance is annotated with a tight bounding box and assigned to its correct class label. Annotations are exported in YOLOv8 format (`.txt` label files with normalised coordinates).

5.3.4 Dataset Split

Split	Proportion	Purpose
Training	70 %	Model weight optimisation
Validation	20 %	Hyperparameter tuning and early stopping
Test	10 %	Final held-out performance evaluation

Table 7: Dataset split summary

5.4 Engineering Calculations

5.4.1 Weighted Item Price Calculation

The price of a weighted product is calculated as:

$$\text{Item Price (ETB)} = \text{Unit Price (ETB/kg)} \times \text{Measured Weight (kg)}$$

The unit price is retrieved from the ERP product record. The measured weight is obtained from the digital scale. The calculation is performed in the backend using Python `Decimal` arithmetic to avoid floating-point rounding errors in monetary computation.

5.4.2 Receipt Total Calculation

The receipt subtotal for each line item is:

$$\text{Line Subtotal} = \text{Unit Price} \times \text{Quantity}$$

The receipt total is the sum of all line subtotals. The backend recalculates all totals after each cashier correction, ensuring the displayed total is always consistent with the confirmed item list. No tax computation is included in the prototype.

6 Implementation

6.1 Hardware Deployment Layout

The prototype hardware is arranged with a camera mounted at a fixed position (overhead or at approximately 45°) above the checkout table surface. The field of view covers the full product placement area. A digital scale is positioned adjacent to the main table for weighted product measurement. A barcode scanner is connected to the cashier workstation. The cashier monitor displays the React interface.

[Insert Figure 6.1: Prototype hardware deployment layout here.]

6.2 Detection Model Training

The YOLOv8 model is trained on the collected and annotated dataset using the Ultralytics training pipeline. Key training configuration:

- **Base model:** YOLOv8n or YOLOv8s (selected based on hardware inference speed requirements).
- **Input image size:** 640 × 640 pixels.
- **Epochs:** 100, with early stopping based on validation mAP.
- **Data augmentation:** horizontal flip, mosaic, colour jitter, and scale variation.
- **Pretrained weights:** COCO pretrained weights used as the starting point (transfer learning).

The trained model weights are exported and integrated into the Django backend as a callable inference function.

6.3 Backend Implementation

The Django backend exposes a REST API that serves the cashier interface and coordinates all system modules. Key API endpoints include:

- `POST /api/detect/` — Accepts a captured image, runs YOLOv8 inference, and returns structured detections with confidence scores and counts.
- `POST /api/checkout/confirm/` — Accepts the cashier-confirmed item list and initiates receipt generation and ERP transaction save.
- `GET /api/products/search/` — Supports barcode or text-based product lookup against the ERP catalogue.
- `POST /api/checkout/weight/` — Accepts a product class and weight value; returns the calculated price.

All product price and name data is retrieved from Odoo via the Odoo XML-RPC API, ensuring billing data comes from the authoritative ERP record rather than the detection model output.

6.4 Cashier Interface

The React cashier interface is organised into three views: (1) the active checkout view, displaying the current draft receipt; (2) the correction panel, providing controls to edit quantities, remove items, or search the product catalogue; and (3) the receipt confirmation view, showing the final receipt before cashier sign-off.

[Insert Figure 6.2: Cashier interface — draft receipt screen here.]

Items detected with below-threshold confidence are visually flagged to prompt cashier attention. The total updates in real time after every correction.

6.5 Odoo ERP Integration

Odoo Community Edition stores product records (name, SKU, barcode, unit price, category, unit of measure) and manages transaction records and receipts. Integration is implemented via Odoo's XML-RPC interface, which allows the Django backend to query and update Odoo records programmatically.

6.6 Source Code Organisation

The project repository is organised as follows:

- `/model` — YOLOv8 training scripts, dataset configuration, and exported model weights.
- `/backend` — Django project including API views, checkout logic, ERP integration layer, weighted product module, and receipt generation.
- `/frontend` — React application including checkout view components, correction panel, receipt preview, and barcode fallback interface.
- `/data` — Dataset images, annotation files, and dataset split manifests.
- `/docs` — Design document, test logs, and supporting materials.

7 Testing and Validation

7.1 Test Strategy

Testing is organised into two layers: model-level evaluation and system-level validation. Model evaluation assesses the performance of the YOLOv8 detection model independently. System validation assesses the correctness of the complete checkout pipeline end-to-end. The two layers are conducted separately because a system can produce correct receipts even when the model is imperfect (due to cashier correction), and a high-performing model does not guarantee correct system behaviour if backend logic or integration has defects.

7.2 Model Evaluation

Model performance is evaluated on the held-out test split using the following metrics:

- **Precision:** the proportion of positive detections that are correct.
- **Recall:** the proportion of true product instances that are detected.
- **mAP@50:** mean average precision at IoU threshold 0.50.
- **mAP@50-95:** mean average precision averaged over IoU thresholds from 0.50 to 0.95 in steps of 0.05.
- **Confusion matrix:** per-class detection and misclassification analysis.
- **Inference speed:** average frames per second on prototype hardware.

[Insert Figure 7.1: Precision-recall curve by class here.]

[Insert Figure 7.2: Confusion matrix — test set here.]

7.3 System Validation

ID	Test Case	Expected Result	Pass Criterion
T-01	Single product detection and receipt	Correct product, price, and total on receipt	Receipt matches expected output
T-02	Multiple products, correct counts	All products detected; quantities correct	Receipt line items match placed items
T-03	Cashier removes incorrect detection	Removed item absent from final receipt	Total recalculates correctly
T-04	Cashier adds missed product manually	Added item present on receipt with correct price	Total recalculates correctly
T-05	Barcode fallback for unknown product	Scanned product added to receipt from ERP record	Correct product name and price retrieved
T-06	Weighted product price calculation	Price = unit price × weight	Calculated price matches manual calculation
T-07	Mixed cart (packaged + weighted)	Both item types on receipt; correct subtotals and total	All prices and total correct
T-08	Quantity change by cashier	Updated quantity reflected in subtotal and total	Arithmetic correct after change
T-09	Transaction saved to ERP	Confirmed transaction visible in Odoo records	Record complete and accurate
T-10	Payment simulation	Payment status recorded correctly	Correct status in ERP record

Table 8: System-level validation test cases

8 Results and Discussion

8.1 Model Performance Results

Metric	Target	Result	Pass/Fail	Notes
mAP@50	≥ 0.80	[TBD]	[TBD]	Evaluate on test split
mAP@50-95	Report	[TBD]	—	Informational
Precision (overall)	Report	[TBD]	—	Informational
Recall (overall)	Report	[TBD]	—	Informational
Inference speed (FPS)	≥ pipeline target	[TBD]	[TBD]	On prototype hardware

Table 9: Model performance summary (to be completed after training)

[Insert Figure 8.1: mAP@50 validation curve over training epochs here.]

This section should be completed after training results are available and should discuss: (1) which product classes showed the strongest and weakest detection performance and the engineering reason; (2) whether precision or recall represents the more significant risk for checkout applications; (3) the relationship between inference speed and hardware choice; and (4) how detection performance compares against the system-level fallback and correction mechanisms.

8.2 System Validation Results

System validation results are to be completed after implementation and testing. For each test case in `@tab:sysval`, this section should report the actual result, whether the pass criterion was met, and any observed failure modes or edge cases. Where a test case failed, the root cause should be identified and the corrective action described.

8.3 Discussion

The discussion should address the following engineering questions based on actual results:

- Does vision-based pre-detection meaningfully reduce cashier workload compared to full manual barcode scanning? What proportion of items are correctly detected without cashier correction?
- What are the most common failure modes (missed detections, wrong class assignments, duplicate bounding boxes), and how does the cashier correction workflow handle them in practice?
- Does the weighted product workflow introduce any latency or usability issues compared to the packaged product workflow?
- Is the prototype hardware sufficient to meet the latency target (≤ 3 s end-to-end), or would deployment require a more capable compute device?
- What is the practical cost estimate for deploying a system of this type in a real retail environment?

9 Conclusion and Recommendations

9.1 Summary

This project designed and prototyped a low-cost, cashier-assisted retail checkout automation system using computer vision. The system uses a YOLOv8 detection model to identify packaged supermarket products from a camera image, a Django backend to match detections against an Odoo ERP product catalogue, and a React interface to enable cashier review and correction before receipt confirmation. Weighted product support, barcode fallback, and digital receipt generation complete the checkout pipeline.

The design prioritises practical deployability in Ethiopian supermarkets: commodity hardware, open-source software, low capital cost, and a cashier-in-the-loop architecture that tolerates model imperfection without producing billing errors.

9.2 Achievement of Objectives

[To be completed after implementation. Assess each specific objective against the test results. State clearly which objectives were fully met, partially met, or not met, with justification.]

9.3 Limitations

- The detection model is limited to product classes present in the training dataset. New products require dataset expansion and retraining.
- Detection accuracy degrades under poor lighting, heavy product overlap, or packaging damage.

- Payment integration is simulated. Full integration with Telebirr or Chapa requires access to live provider APIs, regulatory compliance, and security review.
- Scale reading via OCR is sensitive to scale font, display brightness, and camera angle.
- The prototype has not been validated under real retail load; throughput improvement claims cannot be quantified without field testing.

9.4 Recommendations

- Conduct field testing in a real supermarket environment with actual cashier users before drawing conclusions about throughput improvement.
- Implement confidence-based cashier workflow routing: high-confidence detections auto-confirmed, medium-confidence flagged, low-confidence routed to fallback.
- Build a human-in-the-loop dataset improvement pipeline: log every cashier correction as a training signal and retrain periodically.
- Investigate edge deployment on a low-cost device (Raspberry Pi 5 or NVIDIA Jetson Nano) to produce a concrete hardware cost estimate for retail deployment.
- Expand weighted product support to include OCR-based automatic weight reading from a wider range of scale display types.
- Design an unknown-product detection pathway so that out-of-distribution products are labelled as unknown and routed immediately to fallback rather than misclassified.

J. References

[1] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[2] Odoo S.A., "Odoo Community Edition Documentation," 2023. [Online]. Available: https://www.odoo.com/documentation

[3] Django Software Foundation, "Django Documentation," 2023. [Online]. Available: https://docs.djangoproject.com

[4] React Team, "React Documentation," 2023. [Online]. Available: https://react.dev

[5] T.-Y. Lin *et al.*, "Microsoft COCO: Common Objects in Context," in *Proc. ECCV*, 2014, pp. 740–755.

[6] J. Redmon and A. Farhadi, "YOLOv3: An Incremental Improvement," *arXiv:1804.02767*, 2018.

[Add further references in IEEE format as applicable.]

K. Appendix A: Bill of Materials

Item	Specification	Qty	Estimated Cost (ETB)
Camera (USB webcam or phone)	1080p minimum	1	[TBD]
Digital scale	0–5 kg, 1 g resolution	1	[TBD]
Barcode scanner	USB HID, 1D/2D	1	[TBD]
Checkout table	Plain surface, ≥ 60 × 40 cm	1	[TBD]
Local PC / Raspberry Pi	Ubuntu 22.04 or RPi OS	1	[TBD]
Monitor / display	Minimum 15″	1	[TBD]
Camera mount / tripod	Fixed overhead mount	1	[TBD]
		Total	[TBD]

Table 10: Hardware bill of materials

L. Appendix B: Project Timeline

Activity	M 1	M 2	M 3	M 4	M 5	M 6
Dataset collection and annotation	●	●				
Model training and evaluation		●	●			
Backend development		●	●	●		
Frontend development			●	●		
ERP integration			●	●		
Integration and system testing				●	●	
Report writing and submission					●	●

Table 11: Indicative project Gantt chart (● = active period)

M. Appendix C: Test Logs

[Insert completed system validation test logs here. For each test case in the system validation table, record: the test date, tester name, exact inputs used, observed output, pass/fail result, and any defects raised. Use a consistent tabular format.]